

One Timeline, Many Renderings: A Wolfram Language Paclet for Heterogeneous Musical Output

AuthorA, AuthorB, and AuthorC

InstituteA

Abstract. Algorithmic composition rarely ends in a single medium: the same musical plan may need to become a Csound score for synthesis, engraved notation for performers, a real-time control stream, and a click track for rehearsal. Authored or repaired separately, these timelines drift apart. We present Temporal System, a Wolfram Language paclet that treats heterogeneous output as a projection problem: one immutable store of typed musical entities, placed on a rational beat timeline, is compiled by per-backend renderers into Csound (electroacoustic and synthesis), MusicXML (symbolic score, in beta against a reference schema), and OSC (real-time live electronics) artifacts that stay synchronous because they share one store. We describe the design decisions, the anatomy (temporal, semantic, and rendering layers; fourteen modules), and the renderers. Particular attention goes to Csound, which fixes the rendering idiom: compiled tempo maps yield onset and duration seconds only at render time, session tuning yields p4 frequencies, curve entities compile to k-rate control signals bound to score p-fields, and orchestras remain ordinary `.orc` files addressed by stable named instruments—the score–orchestra separation Csound inherits from the MUSIC N family. A click-track backend derives rehearsal audio from the same meter and tempo and reuses the Csound serializer—how cheaply a further projection follows once the temporal core is shared.

Keywords: Csound, algorithmic composition, music representation, Wolfram Language, click track

1 Introduction

Long before rendering became plural, computing composers had already separated structure from sound. At Bell Labs, Tenney generated note lists for the MUSIC N programs [2,3]; in Utrecht, Koenig’s *Project 1* and *Project 2* computed a piece as tables realized at a later stage—players, the studio, and in his last works Csound itself [1]. In both, the program owns *time and structure* and a separate system owns *sound*: the orchestra–score split Csound institutionalizes [4].

Contemporary practice multiplies the realization stages: a mixed work may need a `.csd` for synthesis, MusicXML for engraving, a timed control stream for live electronics, and a click track for performers. These encode time differently—seconds, divisions per quarter, milliseconds, samples—so maintaining them by hand means maintaining four diverging timelines. Computer-assisted composition environments have long addressed parts of this [5,6]; Temporal System [14] makes one authored timeline the only source of truth and demotes every output file to a derived artifact.

One model feeding many outputs is well established. *music21* exports a single object model to MusicXML, MIDI, Lilypond and text [7]; *abjad* drives Lilypond from a persistent Python score model [8]; MEI encodes a canonical score that Verovio projects to many targets [9,10]; *bach* and *cage* carry a score/timeline abstraction into Max for real-time realization [11]. These are organized around notation or a single real-time host, and in each time is ultimately notational or measured in seconds. Temporal System instead makes one beat-space infrastructure primary: onsets are exact Rationals (musical, not clock, time), driving every layer at once, with conversion to seconds, hertz and samples deferred to each renderer. The host language makes this practical: store, tempo-map inversion, session tuning and per-backend contracts are all Wolfram Language, in which rationals stay exact, transformations compose, and a vast built-in library (including the paclet’s own timeline graphics) sits where the timeline is authored—so authoring the piece and compiling its every projection share one language, not a chain of programs.

Temporal System’s design decisions are few and strict: (i) events live in an immutable store of typed entities whose onsets are exact Rationals in beat space, never seconds; (ii) meter is authored

state, never inferred from durations; (iii) reference tuning is a session parameter, never a constant in conversion code; (iv) the packet generates *scores only*—synthesis instruments remain external `.orc` files, as in Koenig’s and Tenney’s workflows; (v) the shared store is permissive, while each renderer enforces a restrictive contract for its own payload.

One trade-off is worth stating. Authoring the source of truth requires Wolfram Language, a commercial environment, whereas Csound and much of its ecosystem are open-source. The dependency is confined to authoring: the emitted artifacts—the `.csd` with its `#included .orc` files, MusicXML, and the OSC JSON—are plain text in open formats needing no Wolfram runtime to compile, engrave, or dispatch. A free Wolfram Engine covers development and non-commercial use; commercial production still requires a paid license.

2 Anatomy of the Packet

The packet comprises fourteen modules exposing 80 public functions in the three layers of Fig. 1: **core** (entities, store, tempo, session), **rendering** (dispatcher and four backends), and **util** (pitch, rhythm, dynamics, validation, timeline graphics). All functions are pure; store operations return new stores.

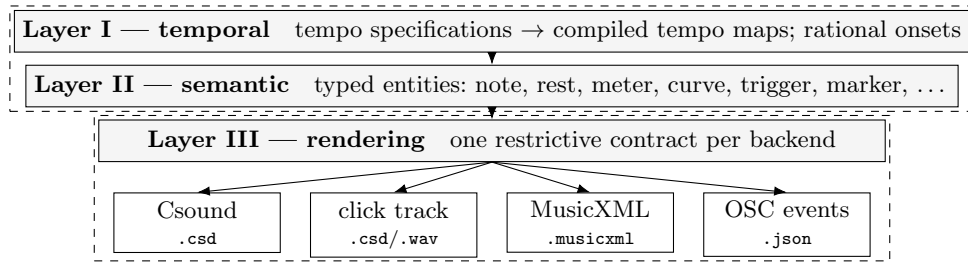


Fig. 1. The three-layer model (dashed groups: Layers I–II backend-agnostic, Layer III and the backends backend-specific). Adding a backend adds a contract in Layer III; Layers I and II never change to accommodate an output format.

Layer I. Tempo is authored as a *tempoSpec*, an ordered list of {beat, BPM} points, and compiled to a piecewise-linear *tempoMap* in {beat, seconds} form with exact analytical inversion. Onsets and durations are Rationals in musical duration space; conversion to seconds or samples happens only inside renderers.

Layer II. The store groups entities by type (`stemData[type][id]`, $O(1)$ typed access) across eleven types (note, rest, trigger, curve, process, state, marker, interval, trajectory, annotation, gesture). A note carries pitch (spelling plus cents, MIDI float, or frequency), duration, dynamic, and a **rendering** association keyed by backend id—the one place backend-specific payloads travel with the entity. Meter is a state entity: measure structure is authored, not guessed. A session configuration carries everything a renderer may not hardcode, starting with `tuningA4` (default 440 Hz, freely reassignable).

Layer III. Constructors validate only the shared shape and require each `rendering[backend]` payload to be an association whose keys belong to the backend contract; malformed Csound payloads are therefore Csound contract errors, local to one renderer, and each backend declares a fallback policy (warn, approximate) for material it cannot express.

The packet also inspects itself: `drawTimeline` renders any `stemData` as lane-based graphics with the metric grid derived from the authored meter entities (Fig. 2), from a single function call with no external tool. Every fixture in the Csound suite renders to a `.csd` that Csound compiles without error, and its rendered audio was auditioned.

3 The Implemented Renderers

A backend is data: an association with an id, a renderer, a serializer, the primitive types it emits, and a fallback policy. The generic dispatcher `renderTo[stemData, tempo, backend, config, registry]`

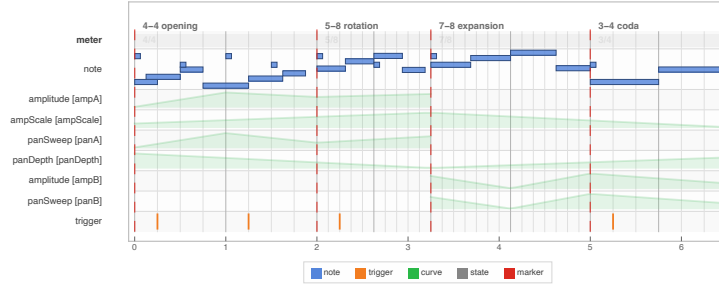


Fig. 2. A timeline exported directly by the packet (`drawTimeline`): the Csound suite’s most complex fixture—sixteen `sine_osc` notes (several with cent offsets), eight `click` hits, four triggers, six overlapping amplitude/pan curves, section markers, and a $4/4 \rightarrow 5/8 \rightarrow 7/8 \rightarrow 3/4$ meter grid. A stress fixture exercising many entity types at once, not a representative composition.

is the single entry point. Four backends exist. *Csound* compiles the whole `stemData` to score and orchestra primitives (Sect. 4); *Click* derives a standalone rehearsal track from meter and tempo alone (Sect. 5). *MusicXML* builds measures from meter state, splits notes at barlines into tied fragments, and validates measure capacity, part coverage, and tie and chord integrity; it is an explicitly lossy projection with a declared coverage profile [12], currently in beta, its export checked by round-tripping through a commercial engraving application for tie, meter and part-separation fidelity. *OSC* emits a schedule-ready JSON event list in milliseconds—curves sampled at a configurable control rate—for dispatch as OSC messages [13] to whatever is listening, a graphical patcher or a running Csound instance through its OSC opcodes. Not every entity exists everywhere: curves become k-rate control signals in Csound but have no notation image; spelling matters to MusicXML, not to synthesis—differences the packet records per entity-backend pair rather than pretending projections are lossless.

4 The Csound Renderer

Csound is the oldest backend and fixes the packet’s rendering idiom, carrying the electroacoustic and synthesis output (Fig. 3). Its p-field convention is stable: `p1` instrument, `p2` onset seconds, `p3` duration seconds, `p4` frequency in Hz, `p5` amplitude in $[0, 1]$, `p6+` from the entity’s `rendering["csound"]` payload. The renderer (`renderCSnd`) takes the whole `stemData`: notes and triggers become `i` statements, markers onset-ordered score comments, and each curve a k-rate control signal in a global variable—an `f`-table reader or a generated `transeg`, per session choice. In the `f`-table form the curve is a GEN-2 table (512 points by default, a session parameter) read at k-rate by `tablei` with linear interpolation.

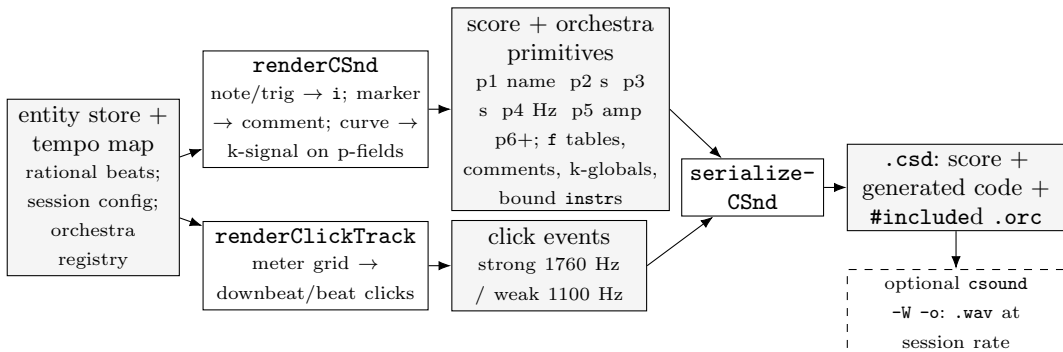


Fig. 3. Both Csound-family renderers share one serializer; source instruments stay in external `.orc` files, and curve bindings add generated copies without touching them.

Two decisions matter most. First, *late conversion*: because onsets stay Rational until this point, `p2/p3` are computed from the compiled tempo map, `p4` from spelling and cents under the session’s

`tuningA4`, and `p5` from the dynamic marking, all at render time. Retuning a session to 432 Hz or editing its tempo specification regenerates every second and every frequency without touching one entity. Second, *named instruments*: an orchestra registry is loaded from a directory of `.orc` files, the filename being the stable id used both in `instr` definitions and in entity payloads. No numeric renumbering ever occurs, so the note

```
i "sine_osc" 0. 4. 440. 0.65 0.5
```

(A4, *mf*, two whole notes at 120 BPM, actual packet output) remains readable and stable as the orchestra grows.

Curves reach the sounding notes through declared bindings: by default an `amplitude` curve multiplies `p5`. For each affected instrument the renderer copies the whole `instr` body from its source `.orc` into a *bound copy*, rewriting only the use-sites of the bound p-fields—there the control-signal globals multiply or replace them, everything else (`pcount()` guards, envelope generation, `limit()` clamps, the oscillator) reproduced verbatim. Affected notes are re-pointed to this copy, and several curves may compose on one p-field: in Fig. 2 two amplitude curves and an `ampScale` curve multiply into `p5`, while `panSweep` replaces and `panDepth` multiplies `p6`. Regenerated from the current `.orc` every render and never edited at the source, the copy cannot drift; locating the use-sites relies on the packet’s p-field convention. The serializer writes the globals and generated instruments beside the registry’s `#include` lines, sorts score statements by onset, and can invoke Csound to compile the `.csd` to audio.

The same amplitude curve lives differently outside Csound. Projected to OSC it is not bound into an instrument but sampled at the session control rate into timed messages. A linear ramp over two seconds at 10 Hz becomes a JSON event list (fields `version`, `sampleRate`, `duration.ms`, and per-sample `events`), each event an OSC message—`time.ms` 1900, `address` `/param/amplitude`, `args` `[0.95]` (actual packet output, excerpted¹). The beat-space curve and tempo map are the same as in the Csound projection; one binds the curve to a score p-field, the other streams it as OSC messages in milliseconds—a genuinely non-Csound artifact, the heterogeneity the architecture exists to make cheap.

5 A Click Track from the Same Timeline

Click tracks for mixed music are usually rebuilt by hand in a DAW whenever the score changes—precisely the divergence this architecture removes. The click backend treats rehearsal audio as one more projection: it reads only the authored meter entities and the tempo map, builds the measure grid, and emits one short `i` event per beat, accenting each downbeat. Every parameter is session configuration—beat unit (1/4), click duration (1/16), strong/weak frequencies (1760/1100 Hz), amplitudes (0.9/0.45), and pan—and the click instrument itself is an ordinary three-line `.orc` file. Because the renderer walks the same meter grid and tempo map as every other backend, meter and tempo changes are followed with no additional authoring; for two 4/4 bars at 120 BPM it emits strong `i` "click" downbeats (0 and 2 s) and weak intervening beats (full `.csd` and `.wav` in the supplementary folder). Reusing `serializeCSnd` unchanged (Fig. 3), the backend inherits `#include` handling and optional `.wav` compilation and needs no change to Layers I–II—showing how cheaply a further projection follows once the temporal core is shared. The genuinely different time encodings, notation’s divisions and OSC’s milliseconds, belong to the MusicXML and OSC contracts.

6 Conclusions

Temporal System contributes an architecture, not a synthesis method: an immutable rational-beat store from which Csound scores, click tracks, notation, and real-time event streams are compiled as unequal but coordinated projections, Csound being the reference renderer that makes the contract concrete. Current work stabilizes the MusicXML backend beyond beta and adds a hierarchical generation layer above the store. Sixty years after *Project 1* and the Bell Labs note-lists, separating computed time from rendered sound still pays—it lets one timeline become many renderings.

¹ Full example artifacts—the aggregated `.csd` and its `.wav`, the click track, the OSC JSON, the generating notebook, and the `.orc` instruments—are in the supplementary folder: <https://drive.google.com/PLACEHOLDER-REPLACE-ME>.

References

1. Koenig, G.M.: Project Two: A Programme for Musical Composition. *Electronic Music Reports* 3, 4–161. Institute of Sonology, Utrecht (1970)
2. Tenney, J.: Computer Music Experiences, 1961–1964. *Electronic Music Reports* 1(1), 23–60. Institute of Sonology, Utrecht (1969)
3. Mathews, M.V.: *The Technology of Computer Music*. MIT Press, Cambridge (1969)
4. Lazzarini, V., Yi, S., Fitch, J., Heintz, J., Brandtsegg, Ø., McCurdy, I.: *Csound: A Sound and Music Computing System*. Springer (2016)
5. Assayag, G., Rueda, C., Laurson, M., Agon, C., Delerue, O.: Computer-Assisted Composition at IRCAM: From PatchWork to OpenMusic. *Computer Music Journal* 23(3), 59–72 (1999)
6. Taube, H.: *Notes from the Metalevel: Introduction to Algorithmic Music Composition*. Taylor & Francis, London (2004)
7. Cuthbert, M.S., Ariza, C.: music21: A Toolkit for Computer-Aided Musicology and Symbolic Music Data. In: *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 637–642. Utrecht (2010)
8. Bača, T., Oberholtzer, J.W., Treviño, J., Adán, V.: Abjad: An Open-Source Software System for Formalized Score Control. In: *Proceedings of the First International Conference on Technologies for Music Notation and Representation (TENOR)*, pp. 162–169. Paris (2015)
9. Hankinson, A., Roland, P., Fujinaga, I.: The Music Encoding Initiative as a Document-Encoding Framework. In: *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 293–298. Miami (2011)
10. Pugin, L., Zitellini, R., Roland, P.: Verovio: A Library for Engraving MEI Music Notation into SVG. In: *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 107–112. Taipei (2014)
11. Agostini, A., Ghisi, D.: A Max Library for Musical Notation and Computer-Aided Composition. *Computer Music Journal* 39(2), 11–27 (2015)
12. Good, M.: MusicXML for Notation and Analysis. In: Hewlett, W.B., Selfridge-Field, E. (eds.) *The Virtual Score: Representation, Retrieval, Restoration*, pp. 113–124. MIT Press, Cambridge (2001)
13. Wright, M., Freed, A.: Open SoundControl: A New Protocol for Communicating with Sound Synthesizers. In: *Proceedings of the International Computer Music Conference*, pp. 101–104. Thessaloniki (1997)
14. Wolfram Research: Wolfram Language and Paclet Documentation. <https://reference.wolfram.com/language/>